

Ordenación de vectores

David Álvarez

1. Introducción

Los algoritmos de ordenación permiten organizar los elementos de un vector de manera ascendente o descendente. A continuación, se presentan los algoritmos de ordenación más comunes, junto con su implementación en lenguaje C.

2. Bubble Sort (Ordenamiento de burbuja)

Este algoritmo va recorriendo el vector y compara elementos adyacentes (el 1º con el 2º, el 2º con el 3º, etc...). Para cada comparación, intercambia los valores del vector si están en el orden incorrecto.

Cada vez que se ha pasado por el vector se coloca, al menos, un dato. Por tanto, en el caso peor, habrá que ejecutar el algoritmo un número de veces igual al tamaño del vector menos 1. Para un vector de 6 elementos, habrá que hacerlo 5 veces.

2.1. Ejemplo

Dado el vector inicial:

mivector = [6, 3, 8, 5, 2, 7]

Vamos a ordenarlo de **menor a mayor**.

Pasos del Algoritmo

1. Primera pasada

- (6,3) → Intercambio → [3,6,8,5,2,7]

- $(6,8) \rightarrow$ No hay intercambio, fíjese que ahora el 2º elemento es el 6, cambió en la comparación anterior.
- $(8,5) \rightarrow$ Intercambio $\rightarrow [3,6,\mathbf{5},\mathbf{8},2,7]$
- $(8,2) \rightarrow$ Intercambio $\rightarrow [3,6,5,\mathbf{2},\mathbf{8},7]$
- $(8,7) \rightarrow$ Intercambio $\rightarrow [3,6,5,2,\mathbf{7},\mathbf{8}]$

Acaba la primera pasada. El mayor elemento (8) ya está en su posición final, pero hay otros elementos descolocados.

2. Segunda pasada

- $(3,6) \rightarrow$ No hay intercambio
- $(6,5) \rightarrow$ Intercambio $\rightarrow [3,\mathbf{5},\mathbf{6},2,7,8]$
- $(6,2) \rightarrow$ Intercambio $\rightarrow [3,5,\mathbf{2},\mathbf{6},7,8]$
- $(6,7) \rightarrow$ No hay intercambio

El segundo mayor (7) ya está en su posición final. Fíjese que, en esta segunda pasada, no es necesario llegar a comparar el último elemento, pues ya estaba colocado. En cualquier caso, si lo hace, el algoritmo funcionará igual puesto que no habrá intercambio.

3. Tercera pasada

- $(3,5) \rightarrow$ No hay intercambio
- $(5,2) \rightarrow$ Intercambio $\rightarrow [3,\mathbf{2},\mathbf{5},6,7,8]$
- $(5,6) \rightarrow$ No hay intercambio

El tercer mayor (6) ya está en su posición final.

4. Cuarta pasada

- $(3,2) \rightarrow$ Intercambio $\rightarrow [\mathbf{2},\mathbf{3},5,6,7,8]$
- $(3,5) \rightarrow$ No hay intercambio

El cuarto mayor (5) ya está en su posición final.

5. Quinta pasada

- $(2,3) \rightarrow$ No hay intercambio

Ya está todo ordenado.

3. Selection Sort (Ordenamiento por selección)

Selection Sort funciona seleccionando, en cada iteración, el elemento más pequeño del vector (o de la a parte no ordenada, si se quiere ser más eficiente) y lo coloca en su posición correcta.

3.1. Ejemplo

Dado el vector inicial:

$$\mathbf{mivector} = [29, 10, 14, 37, 13, 5]$$

Vamos a ordenarlo de **menor a mayor**.

Pasos del Algoritmo

1. Encuentra el menor y colócalo en la posición 0

- El menor elemento en la lista es **5**, intercambiamos con el primer elemento (29).

$$\{5, 10, 14, 37, 13, 29\}$$

2. Encuentra el menor, sin contar el 5, y colócalo en la posición 1.

- El menor en la sublista $\{10, 14, 37, 13, 29\}$ es **10**, y ya está en su sitio.

$$\{5, 10, 14, 37, 13, 29\}$$

3. Encuentra el menor en el resto de datos y colócalo en la posición 2.

- El menor en $\{14, 37, 13, 29\}$ es **13**, intercambiamos con el 14.

$$\{5, 10, 13, 37, 14, 29\}$$

4. Encuentra el menor en el resto de datos y colócalo en la posición 3.

- El menor en $\{37, 14, 29\}$ es **14**, intercambiamos con el 37.

$$\{5, 10, 13, 14, 37, 29\}$$

5. Encuentra el menor en el resto de datos y colócalo en la posición 4.

- El menor en $\{37, 29\}$ es **29**, intercambiamos con el 37.

$\{5, 10, 13, 14, \mathbf{29}, \mathbf{37}\}$

6. Última posición ya está ordenada El vector final ordenado es:

$\{5, 10, 13, 14, 29, 37\}$

3.2. Conclusión

El algoritmo de **Selection Sort** realiza $n - 1$ iteraciones para ordenar el vector. A diferencia de *Bubble Sort*, minimiza el número de intercambios, haciéndolo más eficiente en este aspecto.

4. Otros algoritmos

Existen múltiples algoritmos para ordenar vectores. Cambian en su forma de hacerlo, en las veces que pasan por el vector, el número de intercambio de datos que hacen, etc...

Se han explicado en este documento 2 ejemplos de ordenación que resultan más fácilmente entendibles para un primer curso de programación.

La eficiencia de algoritmos de ordenación es importante e interesante, pero requiere de conocimientos más avanzados.

Si algún alumno le interesa este tema y quiere saber más, puede ponerse en contacto con el profesor.